Deep Learning Fundamentals

A Structured Technical Document for RAG Evaluation

Chapter 1: Introduction to Deep Learning

Deep Learning is a subfield of machine learning that uses neural networks containing multiple computational layers to learn complex patterns from data.

Traditional machine learning often requires manually engineered features. Deep learning models can learn useful representations directly from raw or minimally processed data.

Deep learning is widely used in:

- Computer vision
- Natural language processing
- Speech recognition
- Recommendation systems
- Autonomous systems
- Generative AI
- Medical image analysis

A deep learning system generally consists of:

Input → Neural Network → Prediction → Loss Calculation → Parameter Update

The model repeatedly adjusts its parameters to reduce prediction error.

Chapter 2: Artificial Neural Networks

An Artificial Neural Network (ANN) is composed of interconnected computational units called neurons.

A basic neural network contains:

1. Input layer
2. Hidden layer or layers
3. Output layer

Consider a network receiving three input features:

- Age
- Income

- Credit score

These values enter the input layer.

The hidden layers transform the information using weights, biases, and activation functions.

The output layer generates the final prediction.

2.1 Parameters

The primary trainable parameters of a neural network are:

- Weights
- Biases

During training, these parameters are adjusted to minimize the loss function.

Chapter 3: Neurons and Activation Functions

A neuron calculates a weighted sum of its inputs.

The basic calculation is:

$$\mathbf{z = w_1x_1 + w_2x_2 + ... + w_nx_n + b}$$

where:

- x = input
- w = weight
- b = bias
- z = weighted sum

The weighted sum is passed through an activation function.

3.1 ReLU

The Rectified Linear Unit is defined as:

$$\mathbf{ReLU(x) = max(0, x)}$$

Therefore:

- ReLU(-4) = 0
- ReLU(0) = 0
- ReLU(7) = 7

ReLU is commonly used in hidden layers because it is computationally simple and helps neural networks learn nonlinear relationships.

3.2 Sigmoid

The sigmoid function is:

σ(x) = 1 / (1 + e^-x)

Its output is between 0 and 1.

Sigmoid is often useful for binary classification output layers.

3.3 Softmax

Softmax converts a vector of values into probabilities whose total is 1.

For three classes:

- Cat = 0.70
- Dog = 0.20
- Bird = 0.10

The predicted class would be **Cat**.

Chapter 4: Forward Propagation

Forward propagation is the process of passing input data through the neural network to produce a prediction.

Suppose a neuron receives:

- $x_1 = 2$
- $x_2 = 3$
- $w_1 = 0.5$
- $w_2 = 0.2$
- $b = 1$

The weighted sum is:

z = (0.5 × 2) + (0.2 × 3) + 1

z = 2.6

If ReLU is used:

ReLU(2.6) = 2.6

The resulting value is passed to the next layer.

Forward propagation continues until the output layer produces the final prediction.

Chapter 5: Loss Functions

A loss function measures how different the model's prediction is from the correct target.

The training objective is generally to minimize loss.

5.1 Mean Squared Error

Mean Squared Error (MSE) is commonly used for regression.

$MSE = (1/n)\ \Sigma(y_i - \hat{y}_i)^2$

where:

- y = actual value
- $\hat{y}$ = predicted value
- n = number of observations

If actual values are:

```
10, 20
```

and predictions are:

```
8, 23
```

then:

$MSE = [(10-8)^2 + (20-23)^2] / 2$

$MSE = (4 + 9) / 2$

$MSE = 6.5$

5.2 Cross-Entropy Loss

Cross-entropy is commonly used for classification problems.

For binary classification:

$L = -[y \log(p) + (1-y)\log(1-p)]$

A confident and correct prediction generally produces a smaller loss than a confident incorrect prediction.

Chapter 6: Backpropagation

Backpropagation is the process used to calculate how neural-network parameters contributed to the prediction error.

The process involves:

1. Forward propagation

2. Loss calculation
3. Computing gradients
4. Propagating gradients backward
5. Updating weights and biases

Backpropagation relies on the **chain rule of calculus**.

The gradient indicates how much the loss changes when a particular parameter changes.

A simplified parameter update is:

$w_{new} = w_{old} - \eta \times \partial L/\partial w$

where:

- η = learning rate
- L = loss
- $\partial L/\partial w$ = gradient of loss with respect to weight

---

Chapter 7: Gradient Descent

Gradient descent is an optimization algorithm used to minimize a loss function.

The basic procedure is:

1. Initialize model parameters.
2. Calculate predictions.
3. Calculate loss.
4. Calculate gradients.
5. Update parameters.
6. Repeat.

The update equation is:

$\theta_{new} = \theta_{old} - \eta \nabla L(\theta)$

where:

- θ represents model parameters
- η represents learning rate
- ∇L represents the gradient

If the gradient points toward increasing loss, subtracting the gradient moves the parameters in the direction of decreasing loss.

Chapter 8: Learning Rate

The learning rate controls how large each parameter update is.

A learning rate that is too large can cause training to become unstable.

A learning rate that is too small can make training extremely slow.

For example, consider three possible learning rates:

Learning Rate	Potential Behavior
0.1	Fast but potentially unstable
0.01	Moderate updates
0.0001	Very small updates

The appropriate learning rate depends on the architecture, optimizer, dataset, and training problem.

Common learning-rate strategies include:

- Learning-rate decay
- Step decay
- Cosine decay
- Adaptive learning rates

Chapter 9: Batch Size and Epochs

Batch Size

Batch size represents the number of training examples processed before the model parameters are updated.

Suppose a dataset contains **10,000 examples**.

With a batch size of **100**, approximately:

10,000 / 100 = 100 updates

are performed during one epoch.

Epoch

One epoch means the model has processed the complete training dataset once.

If a model is trained for 20 epochs, it processes the training dataset approximately 20 times.

A larger batch size may provide computational efficiency, while a smaller batch size can introduce more variation into gradient estimates.

Chapter 10: Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are neural networks particularly effective for image-related tasks.

A typical CNN may contain:

Input Image → Convolution → Activation → Pooling → Fully Connected Layer → Output

10.1 Convolution

A convolutional layer applies filters to an input image.

A filter detects patterns such as:

- Edges
- Corners
- Textures
- Shapes

Early CNN layers generally learn simpler patterns, while deeper layers can learn more complex representations.

10.2 Pooling

Pooling reduces the spatial dimensions of feature maps.

Common pooling operations include:

- Max pooling
- Average pooling

For example, a **2×2 max-pooling operation** selects the largest value from each 2×2 region.

If the region is:

```
[1, 5]
[3, 2]
```

the maximum value is:

5

Chapter 11: Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are designed for sequential data.

Examples include:

- Text

- Speech
- Time series
- Sensor readings

Unlike a standard feed-forward network, an RNN maintains information from previous time steps.

A simplified recurrent equation is:

$$\mathbf{h_t = f(W_h h_{t-1} + W_x x_t + b)}$$

where:

- h_t = current hidden state
- h_{t-1} = previous hidden state
- x_t = current input

Traditional RNNs can experience vanishing-gradient and exploding-gradient problems.

LSTM and GRU architectures were developed to improve sequence modeling.

Chapter 12: Transformers

Transformers are neural network architectures that rely heavily on attention mechanisms.

Transformers became highly influential in natural language processing and later expanded into computer vision, audio, and multimodal applications.

A simplified Transformer architecture contains:

- Input embeddings
- Positional information
- Self-attention
- Feed-forward layers
- Residual connections
- Normalization layers

12.1 Self-Attention

Self-attention allows a model to determine how strongly different tokens should influence one another.

The standard attention equation is:

$$\mathbf{Attention(Q,K,V) = softmax(QK^T / \sqrt{d_k})V}$$

where:

- Q = queries

- K = keys
- V = values
- d_k = dimensionality of the keys

Self-attention allows the model to capture relationships between tokens even when those tokens are far apart in a sequence.

Chapter 13: Dropout

Dropout is a regularization technique used to reduce overfitting.

During training, a randomly selected subset of neurons is temporarily ignored.

For example, if the dropout rate is **0.30**, approximately 30% of eligible activations are dropped during each training step.

Dropout encourages the network to avoid relying excessively on specific neurons.

During inference, dropout is disabled and the complete network is used.

Chapter 14: Batch Normalization

Batch Normalization normalizes intermediate activations during training.

It can help:

- Stabilize training
- Improve optimization
- Allow higher learning rates in some cases
- Reduce sensitivity to parameter initialization

Batch normalization generally maintains learnable scale and shift parameters.

It is commonly placed after convolutional or linear transformations and before or around activation functions depending on the architecture.

Chapter 15: Transfer Learning

Transfer learning involves using knowledge learned from one task as a starting point for another related task.

For example, a CNN trained on a large image dataset may already understand basic visual patterns such as:

- Edges

- Textures
- Shapes

A researcher can reuse this pretrained model for medical image classification.

Instead of training the complete network from scratch, the researcher may:

1. Load a pretrained model.
2. Freeze some layers.
3. Replace the final classification layer.
4. Train the new output layer.
5. Optionally fine-tune selected earlier layers.

Transfer learning is particularly useful when the target dataset is relatively small.

Chapter 16: Model Evaluation

Different tasks require different evaluation metrics.

Classification

Common metrics include:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Object Detection

Object detection systems may use:

- Intersection over Union (IoU)
- Precision
- Recall
- Mean Average Precision (mAP)

IoU is calculated as:

IoU = Intersection Area / Union Area

If predicted and ground-truth bounding boxes overlap significantly, IoU will be higher.

Image Segmentation

Segmentation models can use:

- IoU
- Dice coefficient
- Pixel accuracy

The Dice coefficient is:

Dice = 2|A ∩ B| / (|A| + |B|)

Chapter 17: Practical Case Study — Image Classification

A research team develops a deep learning system to classify **12,000 chest X-ray images** into three categories:

1. Normal
2. Pneumonia
3. Other Abnormality

The dataset is divided into:

- 8,400 training images
- 1,800 validation images
- 1,800 test images

The researchers compare three architectures.

Model	Test Accuracy	F1-Score
Basic CNN	81.2%	79.8%
ResNet-based Model	88.6%	87.9%
EfficientNet-based Model	90.1%	89.4%

The EfficientNet-based model produces the highest test accuracy and F1-score.

However, the researchers also examine performance for each individual class because overall accuracy can hide poor performance on minority classes.

Chapter 18: Model Comparison

Model	Primary Use	Major Strength	Major Limitation
ANN	General prediction	Flexible	Poor for spatial structure
CNN	Images	Learns spatial patterns	Computationally expensive

Model	Primary Use	Major Strength	Major Limitation
RNN	Sequential data	Handles sequence information	Can suffer from gradient problems
LSTM	Sequential data	Handles longer dependencies	More complex than basic RNN
Transformer	Language and multimodal tasks	Strong attention mechanism	Computational requirements can be high

---

Chapter 19: Important Formulas

Neuron

$$z = \Sigma w_i x_i + b$$

ReLU

$$ReLU(x) = max(0,x)$$

Sigmoid

$$\sigma(x) = 1/(1+e^{-x})$$

Mean Squared Error

$$MSE = (1/n)\Sigma(y_i - \hat{y}_i)^2$$

Gradient Descent

$$\theta_{new} = \theta_{old} - \eta\nabla L(\theta)$$

Attention

$$Attention(Q,K,V) = softmax(QK^T/\sqrt{d_k})V$$

IoU

IoU = Intersection Area / Union Area

Dice

$$Dice = 2|A \cap B|/(|A|+|B|)$$

Chapter 20: Glossary

Activation Function: A function that introduces nonlinearity into a neural network.

Backpropagation: An algorithm for calculating gradients of neural-network parameters.

Batch: A subset of training examples processed together.

CNN: A neural network architecture commonly used for image processing.

Dropout: A regularization technique that randomly disables activations during training.

Epoch: One complete pass through the training dataset.

Gradient: A vector representing the direction and rate of change of a function.

Learning Rate: A parameter controlling the magnitude of model updates.

LSTM: A recurrent architecture designed to handle long-term dependencies.

Neuron: A computational unit that combines inputs using weights and a bias.

RNN: A neural network architecture designed for sequential data.

Self-Attention: A mechanism that allows tokens to assign importance to other tokens.

Transformer: A neural network architecture based heavily on attention mechanisms.

Transfer Learning: Reusing knowledge from a pretrained model for another task.
